KIMinder360 Tech Feasibility

October 24, 2025

KIMinder360 Team
Dylan Hyer (deh277), Nyle Huntley (nah353),
Cole Bishop (cdb499), Mayanna John (mrj335)
Morgan W. Boatman (Sponsor)
Md Nazmul Hossain (Mentor)

Table Of Contents

Introduction

Context

Modern society faces a growing need for heightened situational awareness. In urban environments, public safety incidents, social disconnection, and lack of preparedness for emergencies are rising concerns. Meanwhile, technology has encouraged increasingly passive engagement, with many individuals spending hours daily on screens that do not cultivate awareness or proactive behavior. A tool that can repurpose this same technology to instill awareness, resilience, and preparedness has the potential to improve both individual and community outcomes.

Problem

Currently, situational awareness training is limited primarily to professional or martial arts contexts, making it difficult for the general population to access. Most existing “wellness” or “fitness” mobile apps focus narrowly on physical activity, meditation, or productivity, but very few incorporate the real-time and randomized situational drills necessary to instill lasting awareness habits.

Our client, Morgan Boatman, is a video producer, author, and owner of *Winter Communications, LLC*. He created *Missions and Madness*, an outdoor team-building experience centered around exploration and developing personal agency. Through his company, Morgan has worked extensively with individuals and groups to improve communication, confidence, and preparedness in unpredictable environments. Building on this philosophy, he developed a series of situational awareness training drills, known as directives, that encourage participants to engage with their surroundings and cultivate readiness in everyday life.

However, Morgan’s current workflow requires manually contacting customers to distribute these directives and track participation. This process is time-consuming, limits scalability, and makes it difficult to maintain consistent engagement. The KIMinder360 mobile app seeks to address these pain points by automating directive delivery and progress tracking, expanding the reach of Morgan’s training system, and transforming it into an accessible digital experience for anyone seeking to improve their situational awareness.

Goal

The KIMinder360 mobile application seeks to close this gap by delivering curated situational awareness drills through smartphones. Users can designate “Action Windows,” choose training categories (e.g., preparedness, fitness, navigation), and

receive randomized directives via push notifications. Progress will be tracked and displayed on a dashboard with end-of-day reviews and performance statistics.

Key features include:

- User customization of training windows and directive categories.
- Randomized push notifications with actionable tasks.
- Secure storage of user preferences, progress, and performance data.
- Progress dashboard with analytics and general information about completed objectives.

Impact

By gamifying situational awareness, the KIMinder360 mobile app will allow anyone to learn valuable preparedness skills applicable in both daily life and emergencies. Its mobile app format dramatically expands reach and accessibility, turning self-improvement into an accessible, fun, and engaging experience that anyone can participate in.

Now that the importance of KIMinder360 has been established, we will turn to the technological feasibility of implementing the project. At this early stage, we are analyzing the key technological challenges, identifying possible alternatives, and selecting the most promising solutions. This document identifies the core technological challenges, explores alternative solutions, and outlines the preliminary choices best suited to ensure the app's success.

Technological Challenges

With the goals and impact of the KIMinder360 project established, the next step is to assess the technical feasibility of implementing the app. Our development approach needs to balance Morgan's vision for a practical, accessible training tool with the requirements for scalability, performance, and long-term maintainability. To that end, we have identified several high-level technological challenges that will shape our design decisions and overall development plan.

1. **Android Native Development:** Selecting the most effective platform for native Android development is a top priority. Because Morgan wants this application to be developed for Android as our first priority, choosing a framework that supports efficient, intuitive Android deployment will allow us to accelerate initial development. Ideally, the platform should provide strong tooling support, minimize manual setup, and enable smooth integration with backend services.

2. **Cross-Platform Development:** Once the Android version is complete, Morgan envisions expanding KIMinder360 to additional platforms like iOS as a stretch goal. To prepare for this, we must select a cross-platform development tool and programming language that facilitates future scalability. This will ensure that when development extends to iOS or other platforms, feature parity and consistent performance can be maintained with minimal rework.
3. **Scalable Database:** A secure and scalable database is essential for storing directives, categories, and difficulty levels. Because the app's core functionality relies on accessing and managing a structured collection of directives, the chosen DBMS must support efficient queries and flexible schema evolution. Additionally, the database should integrate seamlessly with the mobile development environment to support our own curated data as well as user-generated content.
4. **Cloud Hosting Solutions:** To balance offline functionality with community-driven online features, such as shared directive lists and optional location-based services, we must identify a reliable cloud hosting solution. The chosen provider should offer scalable backend support, smooth API integration, and robust data synchronization with our offline database. This will allow KIMinder360 to function primarily offline while still providing periodic cloud connectivity for updates and community content retrieval.

Technology Analysis

Developing this application required careful consideration of multiple technical challenges, each with its own set of desired characteristics and potential solutions. This section provides an in-depth look at the technological decisions that shaped our development process. We begin by identifying the specific challenges our app presents, ranging from building a native Android experience and achieving cross-platform compatibility to managing local data storage and ensuring scalable cloud integration. For each challenge, we define the technical requirements, explore several alternative tools or frameworks, and systematically analyze their strengths and limitations. The analyses were conducted using a comparative matrix approach, evaluating each technical alternative against the specific characteristics we wanted. Based on this structured evaluation, we present our chosen technologies and justify each selection with clear reasoning and feasibility validation, demonstrating how our technology stack effectively supports both current goals and future scalability of the application.

Challenge 1: Android Native Development

Issue: The app must be developed natively for Android to take advantage of system-level features like push notifications, alarm scheduling, and offline persistence. Multiple development environments exist that can create mobile applications, but we face the challenge of selecting one that will allow us to create the most refined Android experience easily.

Desired Characteristics:

- **Push Notifications**: The core functionality of this app requires easy access to building notifications and sending them out to the user's phone. The intent of the KIMinder360 app is to send the user notifications containing directives, so being able to send notifications through Android with as little resistance as possible is crucial.
- **Alarm Scheduling**: To allow users to schedule "active hours" and for us to send notifications randomly within those hours, we have to schedule alarm notifications on the phone.
- **Offline Persistence**: Since this app is primarily intended to be an offline experience, we will have to store data on the phone's local storage so that it persists even after the app is closed. A development environment where we can create local databases and store data will be crucial.
- **Access to Sensors Like GPS**: Potential stretch goals and features down the line, like location tracking, mean that being able to tap into the phone's sensors and various features is crucial.
- **Strong Documentation and Community Support**: To accelerate the development process, we want a platform that has a large community willing to support new developers and strong documentation that can be used to learn about the platform and aid development.
- **Long-term maintainability**: A language that has a strong future of being relevant in Android development so that development can continue with modern features.

Alternatives

- **Godot** - Primarily a 2D/3D game engine with limited Android system-level integration. Best suited for interactive or gamified apps, not for full-featured native applications.
- **Android Studio** - The official IDE for Android development. Provides full access to all Android APIs, Jetpack libraries, and Kotlin language support.

- **Unity** - Another game engine with cross-platform support, but it lacks direct access to Android-specific APIs without complex plugin setups. Primarily for game development rather than general-purpose apps

Analysis

Feature	**Godot**	**Android Studio**	**Unity**
Push Notifications	⚠️	✅	⚠️
Alarm Scheduling	⚠️	✅	⚠️
Offline Persistence	⚠️	✅	⚠️
Access to Sensors like GPS	⚠️	✅	⚠️
Documentation and Community Support	✅	✅	✅
Long Term Maintainability	❌	✅	❌

Key: ✅ = Works Natively, ⚠️ = Requires Workarounds, ❌ = Not Supported

Chosen Approach:

For our chosen approach and solution to this challenge, we have gone with **Android Studio**, the official and most robust choice for native Android app development. We have decided to go in this direction because it meets all the desired characteristics we have outlined for a native Android development environment. Android Studio provides direct access to essential Android system features such as notifications, background tasks, and phone sensors, allowing us to fully leverage the capabilities of the platform. Integration with Room database means that we will be able to take full advantage of database features and persistent local storage capabilities. It also offers first-class Kotlin support, the preferred language for Android development according to Google, which enables us to write modern, maintainable, and safe code. Additionally, because it is backed by Google's long-term development roadmap, Android Studio ensures stability, regular updates, and compatibility with future Android versions. Finally, because it is the primary development environment for all Android apps, that means there is a large community and plentiful documentation provided not only by Google but third-party sources as well, so that we can always reach out for assistance when development roadblocks are reached.

Proving Feasibility

A simple Hello World application was developed in Android Studio with Kotlin to prove the feasibility of Android Studio as our chosen solution. This app was set up to display a simple screen reading “Hello World” to show how we could put together an app that boots up and shows content to the user. It was also given 2 pieces of functionality: the ability to request the user’s location and the ability to build and send notifications over a custom notification channel. These two features prove that we can build, schedule, and send out notifications to the user as well as gain access to phone sensors and information like location. This demo app can be found in our public GitHub repository.

Challenge 2: Cross-Platform Compatibility (Android & iOS)

Issue: As a stretch goal, the app should run seamlessly on both Android and iOS platforms, maintaining identical functionality and appearance. However, this goal is secondary because our client, Morgan, primarily uses an Android device and wants the app to first be fully functional on that platform. Once a stable Android version is complete, we intend to explore extending compatibility to iOS to broaden accessibility and long-term usability.

Desired Characteristics:

- **Easy to learn**: The chosen framework should be straightforward for our team to adopt, minimizing onboarding time and allowing more focus on development and feature refinement.
- **Effective Code Translation**: Ideally, the framework should integrate with our Android Studio environment and allow Android code to be reused efficiently for iOS builds.
- **Time Efficient**: The solution should minimize redundant work and simplify the process of maintaining a shared codebase across both platforms.

Alternatives:

- **Swift Development Environment** - Apple's native language for iOS app development. While powerful, this approach would require developing two separate apps in Kotlin for Android and in Swift for iOS, effectively doubling time and effort.
- **Kotlin Multiplatform Mobile** - A Kotlin-based framework integrated into Android Studio that allows developers to share code between Android and iOS projects. KMM is used in many production environments and aligns well with our existing Kotlin workflow.
- **Flutter** - Google's open-source UI toolkit for cross-platform development. Although not natively integrated into Android Studio's Kotlin ecosystem, it provides a unified framework for building both Android and iOS apps.

Analysis:

Feature	**Swift**	**Kotlin Multiplatform**	**Flutter**
Easy to learn	❌	✅	⚠️
Effective code	❌	✅	✅

translation			
Efficient implementation	❌	✅	⚠️

Key: ✅ = Meets Expectations, ⚠️ = Partially Meets Expectations, ❌ = Does Not Meet Expectations

Chosen Approach:

Our preferred long-term solution is **Kotlin Multiplatform Mobile (KMM)**, as it aligns directly with our use of Kotlin and supports efficient, shared codebases across Android and iOS. However, KMM's current limitation is that its iOS build functionality is not yet fully supported on Android Studio for Windows, which is the environment most of our team uses. While the developers behind KMM are actively expanding Windows support, this limitation prevents us from adopting it for our immediate development phase.

For now, we intend to use **Flutter** as our cross-platform development framework. Although it is less tightly integrated with Kotlin, Flutter provides reliable cross-platform capabilities and can be implemented on Windows systems without compatibility issues. While it does involve more setup and learning than Kotlin Multiplatform, it is currently the most practical and accessible option for our team.

Proving Feasibility:

To prove the feasibility of Flutter for cross-platform development, we created a demo project using Flutter within Android Studio to confirm compatibility with our chosen environment. The demo successfully ran on our development machines, demonstrating that Flutter integrates cleanly with Android Studio and can serve as a viable solution for future iOS compatibility. Our demo project is hosted publicly on GitHub.

Challenge 3: Database and Data Storage

Issue: A scalable database is essential to KIMinder360's functionality, as it stores and manages all directive data, including categories, difficulties, and user preferences, while ensuring complete offline access. Because the app's core experience relies on delivering randomized training directives without requiring an internet connection, the database must be lightweight, reliable, and tightly integrated with Kotlin. It should efficiently handle local queries and updates while maintaining data security and integrity. Although future scalability is a consideration, the primary goal is to ensure a stable offline data structure that provides consistent performance and a seamless user experience on Android devices. Critically, the directive data serves as the foundation of KIMinder360's functionality and brings the app's situational awareness training to life.

Desired Characteristics:

- **Kotlin Compatibility:** The database must integrate cleanly with Kotlin to simplify implementation, reduce boilerplate code, and ensure long-term maintainability within Android Studio's native development environment.
- **Offline Use:** The app must fully function without internet access, requiring a local database capable of quickly retrieving, updating, and persisting directive data entirely offline.
- **Secure Data Protection:** User preferences and progress tracking data must be protected through encryption or controlled access mechanisms to maintain user privacy and trust.
- **Free to Use:** To support long-term sustainability, the chosen database solution should be open-source or included in Android's native tools, avoiding usage fees or subscription costs over time.
- **Long-term Support:** The database should be backed by an active developer community or official support channel, ensuring continued compatibility with future Android updates and reliable documentation.
- **Cloud Sync:** As a stretch goal, the database should be structured in a way that allows future integration with cloud synchronization or user-shared content without major architectural changes.

Alternatives:

- **Room** - Google's abstraction layer for SQLite, Room belongs to the Android Jetpack suite. It has been in production use since 2018, is fully open source, and has documentation and long-term support from Google. Primarily used in purely local Android apps.
- **Firestore** - Google's cloud-hosted NoSQL database belonging to the Firebase suite. It provides real-time synchronization across devices and offline caching. It

has been in production use since 2019, typically in apps that require cloud sync or shared, dynamic content.

- **Realm** - Acquired by MongoDB in 2019, initially released in 2017, Realm is an open-source and object-oriented database system. It is commonly used as a replacement for SQLite-based DBs, particularly for apps that are user-data intensive or have complex local object graphs.

Analysis:

Feature	**Room**	**Firestore**	**Realm**
Kotlin Compatibility	✅	⚠️	⚠️
Offline Use	✅	⚠️	✅
Secure Data Protection	✅	✅	✅
Free to Use	✅	⚠️	⚠️
Long-term Support	✅	✅	❌
Cloud Sync	❌	✅	⚠️

Key: ✅ = Full support, ⚠️ = Moderate support, ❌ = Not supported

Chosen Approach:

Room was selected as the most suitable option due to its native Kotlin integration, strong offline support, and reliable long-term maintenance within Android Jetpack. It offers a secure, free, and well-documented solution that aligns perfectly with the app's need for local data persistence and scalability potential. While Firestore provides built-in cloud sync and Realm offers reactive data handling, both introduce added complexity and cost. Room's simplicity, stability, and deep Android integration make it the best foundation for initial development, with the option to expand later using Firestore for cloud-based sharing, as discussed later in the Cloud Hosting and Scalability section.

Proving Feasibility:

To verify the feasibility of Room for data storage, we've created a test app with Room implementation that retrieves a random entry from ~100 data entries, acting as the directives to be added. We've confirmed that Room is able to efficiently handle queries relating to directive categories and difficulties, supporting fully offline access, and cleanly integrating with Kotlin. Our demo app is hosted publicly on GitHub.

Challenge 4: Cloud Hosting and Scalability

Issue: The backend must store and deliver directives and categories, manage user accounts, and record user progress as the system scales. The initial version of the app will include built-in directives, but as a stretch goal, users will be able to contribute their own. Therefore, the cloud hosting platform must be able to maintain stable performance under high user activity and data growth.

Desired Characteristics:

- **Cost-effectiveness:** To support our initial deployment and testing, our chosen cloud hosting platform must be able to include free or low-cost hosting tiers.
- **Scalable:** The cloud hosting system can handle user growth, as well as increased data, without loss of performance.
- **Ease of Integration:** For our early stage of development, the focus of integration will be Android Studio. As we expand later, our chosen cloud system must provide easy integration with other platforms to ensure smooth scalability.
- **Low Setup Complexity:** Allows for the team to deploy and manage the backend easily without needing a lot of in-depth or advanced experience in that area of development.

Alternatives:

- **Firebase (Firestore, Auth, Messaging)** - Google's Firebase provides a readily available infrastructure, which allows us to focus on developing the app rather than on the back-end frameworks. NoSQL, authentication, and push notifications are integrated into one.
- **AWS Amplify** - Configuring Amazon's AWS Amplify serves as the blueprint for linking front-end and back-end services. We would be able to define APIs, data models, and authentication settings by using a simple syntax in the configuration file.
- **Supabase** - Supabase's simple backend service offers an easy-to-use dashboard, PostgreSQL database, and auto-generated APIs. This platform is beneficial for managing user profiles and the directives records.

Analysis:

Feature	**Firebase**	**AWS Amplify**	**Supabase**
Cost-Effective	✅	⚠️	✅
Scalable	✅	✅	⚠️

Easy Integration	✅	⚠️	⚠️
Setup Complexity	⚠️	❌	✅

Key: ✅ = Meets Expectations, ⚠️ = Requires Workarounds, ❌ = Not Supported/Overly Complex

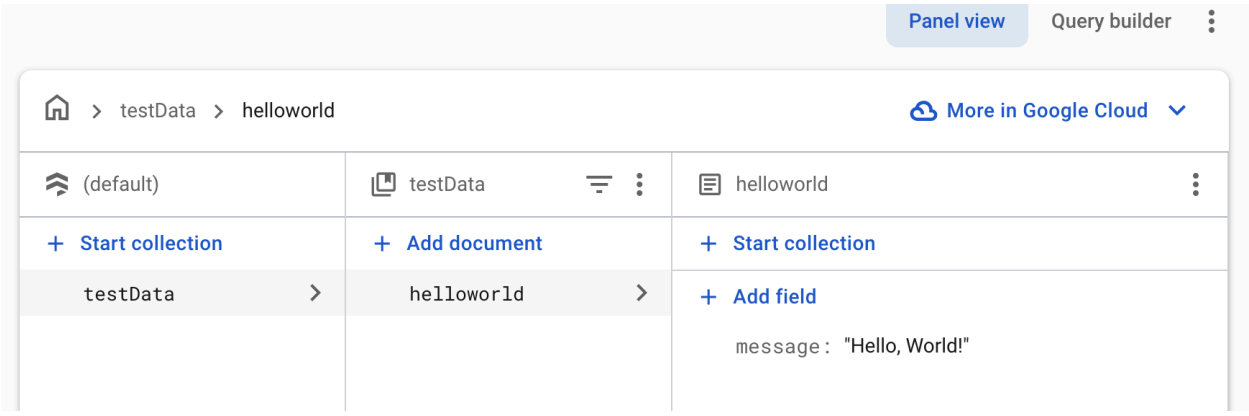
Chosen Approach:

Firebase suite (Firestore + Auth + Messaging)

Firebase was the chosen platform since it provides an accommodating balance between cost, scalability, and ease of setup for our project. Its database supports data management in real-time through Firestore, which would allow quick updates across all devices. This real-time feature goes hand-in-hand with cloud messaging to help deliver notifications that are timely. AWS Amplify generally requires experience with AWS products. While Supabase does provide real-time features, its performance is less stable than Firebase, which is unsuitable for our project's need for continuous live updates.

Proving Feasibility:

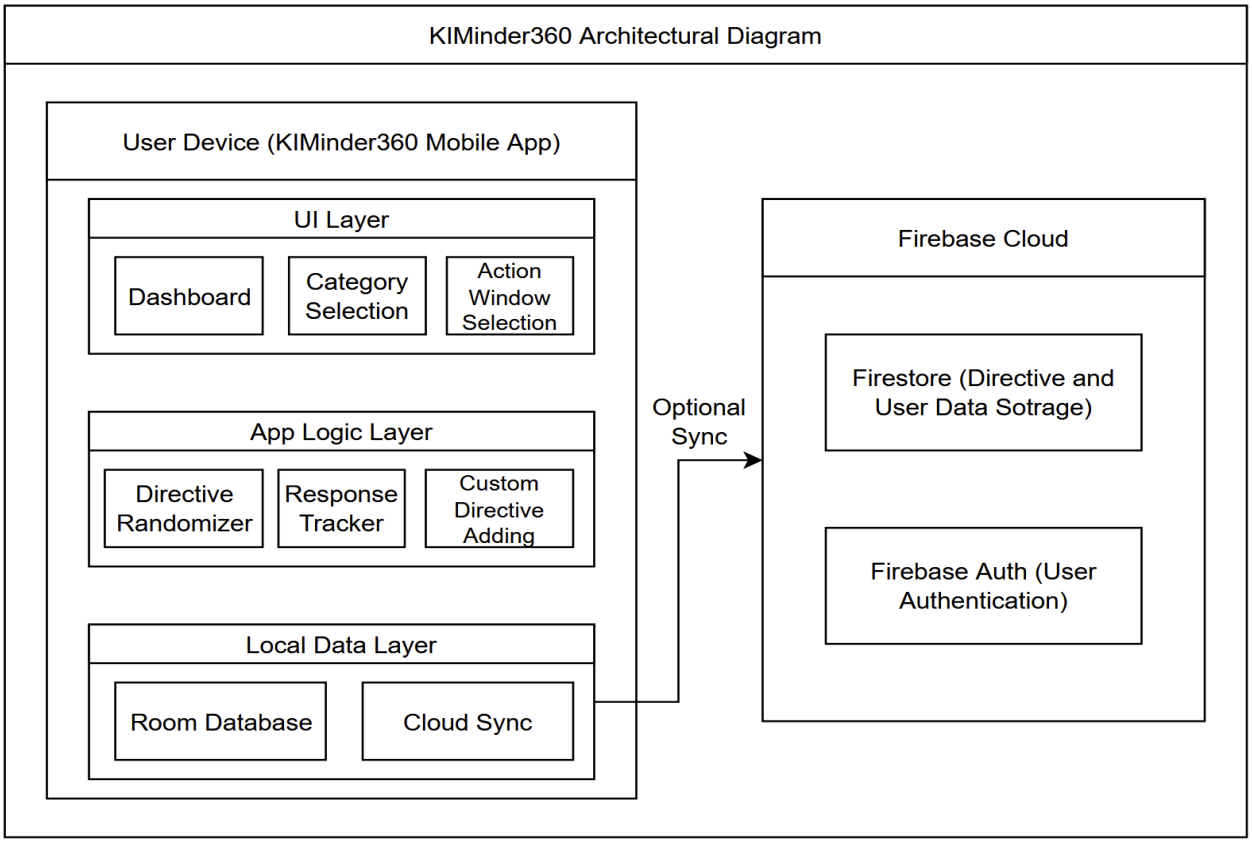
To prove the feasibility of Firebase for cloud hosting and scalability, we've created a "Hello, World!" test using Firebase Firestore. A new project was set up in the Firebase menu, and a sample collection, titled "testData," was created, which contained the document "helloworld" with the field message = "Hello, World!". With this test, we've confirmed that Firebase can store and retrieve data in real-time.



Technology Integration

Having explored the primary development challenges of KIMinder360, including Android-focused design, cross-platform expansion, offline-first data storage, and potential cloud connectivity, we've established clear, practical solutions for each. Our next step is to validate these approaches through a prototype that combines our chosen technologies. This integration phase will not only demonstrate technical feasibility but also confirm that our planned architecture supports scalability and user engagement as intended.

We will integrate the proposed solutions into a unified demo application that validates the feasibility of our technical approach. A base Android app will be developed in Android Studio to establish the functionality of Android studio for development. That app will be adapted to use Kotlin Multiplatform so that we know our plan for cross-platform development is feasible. We will then create a Room database for our app to ensure that basic tables and request operations are working and that we can make a scalable DB. Finally, we will make a simple cloud integration demonstration that works with our demo Android app. Through these steps, we will ensure that each of our solutions is compatible and integrates well with one another. Below is an architectural diagram that illustrates how our chosen solutions will integrate with each other:



Conclusion

The KIMinder360 project aims to close a critical gap in situational awareness training by delivering a mobile platform for real-time, accessible drills. In this feasibility analysis, we identified the main technical challenges and developed strategies to overcome them, particularly in the areas of cross-platform development, offline data management, and potential cloud expansion. In the end, our research led us to decide on using Kotlin and Android Studio for initial development, with Kotlin Multiplatform to handle cross-platform compatibility in the future. For our database integration, we will use Room for local storage and Firebase Firestore for cloud storage. Our chosen solutions to these technical challenges make us confident in the future development of this project.

Research:

Major Advantages of Android Studio

Get started with Android

What is Android Studio and Why Should You Use It?

Why Android Studio Is Better For Android Developers

A Complete Guide to Learn Android Studio

Native vs cross platform development

Kotlin and Android

Flutter vs swift

How Firebase is Emerging as an Innovation in App Development

Firebase vs. AWS Amplify: A Practical Comparison for Flutter Development